

MACHINE LEARNING



PERFORMANCE ANALYSIS OF CARPOOLING OPTIMIZATION SYSTEM

Team:

DHONDI VARUN: 22MIA1200

KASHISH GIDWANI: 22MIA1117

CODE LINK:

<https://colab.research.google.com/drive/1x83qgq7UoMYIdwQKMbPLQp3wzdrteVzu?usp=sharing>

1. Introduction

The code focuses on solving a carpooling optimization problem by leveraging machine learning models and linear programming. The primary goals are to:

- Optimize routes for students with cars (drivers) and passengers.
- Minimize costs associated with fuel and time constraints.
- Ensure scalability and efficiency.

The notebook integrates several advanced techniques, including K-Means clustering, hierarchical clustering, XGBoost classification, and operations research methods like Linear Programming.

2. Problem Breakdown

Objectives:

1. Identify drivers and passengers based on dataset attributes.
2. Group drivers and passengers geographically and temporally.
3. Predict feasible driver-passenger matches.
4. Minimize travel costs using fuel price and route optimization.

Datasets:

1. Carpooling Dataset:

- Attributes: Driver/Passenger location, time of availability, and vehicle details.
- Purpose: Identify potential drivers and passengers.

2. Petrol Prices Dataset:

- Attributes: Date and price per liter.
 - Purpose: Estimate fuel costs for routes.
-

3. Code Structure and Functionality:

3.1 Data Loading and Preprocessing

- **Dataset Handling:**
 - carpooling_data and petrol_prices_data are loaded using pandas.read_csv().
 - Columns like time and date are converted to proper formats for filtering and merging.
 - **Observations:**
 - File paths are hardcoded, reducing portability.
 - Missing handling for exceptions in file reading.
 - **Recommendations:**
 - Implement dynamic file path handling (e.g., via os or user inputs).
 - Add error handling for file reading.
-

3.2 Clustering

- **Techniques Used:**
 - **K-Means:** Groups passengers and drivers by location proximity.
 - **Hierarchical Clustering:** Produces dendrograms for deeper insights into passenger-driver clusters.
- **Strengths:**
 - Logical grouping ensures drivers have nearby passengers, reducing fuel consumption.
- **Weaknesses:**

- No hyperparameter optimization (e.g., finding the optimal number of clusters using the Elbow Method).
 - Computational cost of hierarchical clustering for large datasets.
 - **Metrics Evaluated:**
 - Silhouette Score: Measures cluster cohesion and separation.
 - **Improvements:**
 - Add automatic cluster evaluation methods (Elbow, Gap Statistics).
 - Use optimized algorithms for hierarchical clustering (e.g., fast linkage methods).
-

3.3 Classification

- **Model:**
 - XGBoost classifier predicts feasible matches between drivers and passengers.
- **Features:**
 - Location, time overlap, fuel consumption, and car type.
- **Metrics:**
 - **Classification Report:** Evaluates precision, recall, and F1-score.
- **Strengths:**
 - XGBoost's inherent ability to handle imbalanced datasets and outliers.
- **Weaknesses:**
 - Lack of cross-validation.
 - Missing feature importance analysis.
- **Recommendations:**
 - Use cross-validation to avoid overfitting.
 - Plot feature importance to understand model decisions.

3.4 Linear Programming

- **Methodology:**
 - Binary variables represent driver-passenger assignments.
 - Constraints ensure:
 - Each driver picks up at most three passengers.
 - Routes minimize fuel costs.
- **Strengths:**
 - Combines machine learning results with mathematical optimization.
- **Weaknesses:**
 - Limited scalability due to exponential growth in constraints for large datasets.
- **Recommendations:**
 - Explore heuristic or metaheuristic algorithms (e.g., Genetic Algorithms) for large-scale optimization.

3.5 Cost Analysis

- **Implementation:**
 - Combines fuel price data with travel distances to calculate cost per passenger.
 - Uses clustering results for route optimization.
- **Visualizations:**
 - Histograms for cost distribution.
 - Line plots for fuel consumption trends.
- **Observations:**

- Effective use of petrol price data for real-world relevance.
 - **Suggestions:**
 - Add variance analysis for costs across clusters.
 - Normalize cost metrics to handle outliers.
-

4. Performance Metrics and Analysis

4.1 Clustering

- **Metric:** Silhouette Score.
 - Strength: Measures cluster quality effectively.
 - Weakness: Does not consider cluster size imbalance.
- **Proposed Enhancements:**
 - Use additional metrics (e.g., Davies-Bouldin Index).

4.2 Classification

- **Metrics:**
 - Precision, Recall, F1-score from the classification report.
 - No evidence of confusion matrix analysis.
- **Suggestions:**
 - Include a confusion matrix for error analysis.
 - Compare with simpler models to validate XGBoost's choice.

4.3 Optimization

- **Objective Function:**
 - Minimize travel costs while adhering to constraints.
- **Scalability:**
 - Time complexity increases with the dataset size.
 - Missing runtime and memory usage evaluations.

- **Suggestions:**
 - Implement profiling tools (e.g., cProfile) for runtime analysis.
-

5. Visualization and Reporting

- **Strengths:**
 - Clear and concise histograms and dendrograms.
 - Suitable use of seaborn and matplotlib for reporting.
 - **Weaknesses:**
 - Lack of annotation in dendrograms for cluster interpretation.
 - Missing correlation analysis between fuel costs and passenger assignment.
 - **Recommendations:**
 - Include heatmaps for cost and cluster correlation.
 - Add dynamic visualizations (e.g., plotly) for interactive analysis.
-

6. Scalability and Limitations

6.1 Scalability

- **Clustering:** Computational overhead grows with dataset size.
- **Optimization:** Constraint-solving complexity may hinder scalability.

6.2 Limitations

1. **Portability:**
 - Hardcoded paths restrict usability across environments.
2. **Error Handling:**
 - Minimal exception handling for missing or malformed data.

3. Reproducibility:

- Missing seeds for random processes in clustering and classification.

7. Recommendations for Improvement

1. Data Preprocessing:

- Automate data validation and cleaning steps.
- Use modular functions to preprocess both datasets.

2. Model Selection and Evaluation:

- Compare models like Random Forest with XGBoost.
- Perform hyperparameter tuning for clustering and classification.

3. Optimization Scalability:

- Replace linear programming with heuristic algorithms for larger datasets.

4. Metrics:

- Introduce runtime, memory usage, and algorithmic complexity evaluations.

5. Documentation:

- Add detailed comments in code cells for reproducibility.
- Provide a summary table for all key metrics.

7. Conclusion

This notebook provides a robust framework for optimizing carpooling using machine learning and operations research. Despite its strengths, addressing scalability, reproducibility, and code efficiency can significantly enhance its practical applicability.